DOINGG@UNION.EDU

COURSE: CSC106 CAN COMPUTERS THINK?

SELF

Contents

Home

The course website for CSC106, part of the Union College CS curriculum.
Here you can find our weekly schedule, assignmnets and resources.

Announcements

Office Hours

- Student/Office Hours:
  - Steinmetz 217
  - Monday 4:00 pm - 5:30 pm

- Thursday 3:00 pm - 4:30 pm
- subject to change, check course website for most up-to-date schedule
- drop-in or schedule a 15 minute slot: https://calendar.app.google/8bus6pfDvyphR9ar5
- by appointment for another time or over zoom

Course Description

Can computers think? Is it possible to build intelligent machines? What does it mean for a machine to think or to be intelligent? Why, or why not, would we want intelligent machines? To what extent do we already have them? How do they work? How do they impact our lives? What are their limitations? This course invites you to explore these questions and more.

This course is an introduction to the field of computer science with an artificial intelligence theme that does not assume you have prior experience with computer programming or computer science. It introduces basic algorithms, data structures and programming techniques. You will learn how computer scientists think about and solve problems by doing your own computer programming. This will give you an understanding of how the current technology for building intelligent machines works. In addition, you will read non-technical papers about artificial intelligence and the ethical questions raised by its use. You will reflect on the ways current computational systems often reinforce biases and can harm groups of people inequitably. And you will evaluate how the choices you are making when creating your own programs will affect future users.

If you're thinking about a CS or CPE major or minor, this course will give you a solid foundation to continue to *Programming on Purpose* (CSC 120), to *Data Structures* (CSC 151) and beyond. If you're a neuroscience major, it will help you see how scientists are using biological and neurological principles to model "behavior'' in computers and it will teach you skills useful for processing experimental data. If you are thinking about another major or minor, this course will give you foundations to apply computing in whatever area you are interested in. And if you're here just because you're curious, that's great!

Schedule

Weekly Schedule

Schedule

check often as things may change

W	TOPIC	Due by end of week	Runestone
1	Programming, Syntax and Semantics	Portfolio (PF) 1	1 & 2
2	Variables and Functions	PF 2	3 & 4
3	Conditional Statements	PF 3, Project Demo 1	5
4	Conditional Loops	PF 4, Practical Exam 1	6
5	Counting Loops	PF 5, Project Demo 2	7
6	Strings, Lists	PF 6, Written Exam 1	8 & 9
7	Nested Lists, Dictionaries	PF 7, Project Demo 3	10
8	File I/O	PF 8, Practical Exam 2	11
9	Recursion	PF 9, Project Demo 4	
10	Searching, Sorting	PF 10	

W	TOPIC	Due by end of week	Runestone
11	Finals Week Monday 11/23 2:30 - 4:30	Written Exam 2	

Resources

CS-Ticket

- technical help with lab machines
- https://csticket.union.edu/

Runestone

- https://runestone.academy/runestone/default/user/register
- your Union College email
- unioncollege_foppff_fall26 as the course name

Gradescope

- https://www.gradescope.com/courses/1370118
- Entry Code: 3ZGEVN

Late Token Form

- https://forms.gle/bKTWFgftoT9sYFxJ7
- fill out this form to get a 24 hour extension on any Programming Project
- each student gets 3 tokens per term

Python

- Python 3.13.X: the software package needed to write and run Python programs
- https://www.anaconda.com/download/success?reg=skipped

Python Documentation

- Official Documentaion: https://docs.python.org/3/index.html
- Can *always* be used as a reference
- Can be searched for module-specific documentation e.g. turtle

Reeborg

- Reeborg's World: a useful website for learning some programming basics
- Reeborg's World: https://reeborg.ca/reeborg.html
- Reeborg Project Documentation

PythonTutor

- Python Tutor: a useful website for tracing python code
- https://pythontutor.com/visualize.html#mode=edit

Google Colab Pro

- Pro account sign-up: https://colab.research.google.com/signup
- Use your Union email
- A pro account is not needed for this course, just an option Union pays for
- Notes template: https://colab.research.google.com/drive/15rPmKyQKCCbwsdDCpC39o8hczWZ-B5zV?usp=sharing

Python Style Guide

- PEP 8
- should be used in all code cells of Colab notebooks
- Link: https://peps.python.org/pep-0008/

Syllabus

Syllabus

- check for updates! (All updated text in this posted version of the syllabus will supersede text from older versions.)

CSC106 Syllabus

1 CSC 106: Can Computers Think?

- Union College, Fall 2026
- Georgia Doing, PhD: email: doingg@union.edu, office: Steinmetz 217

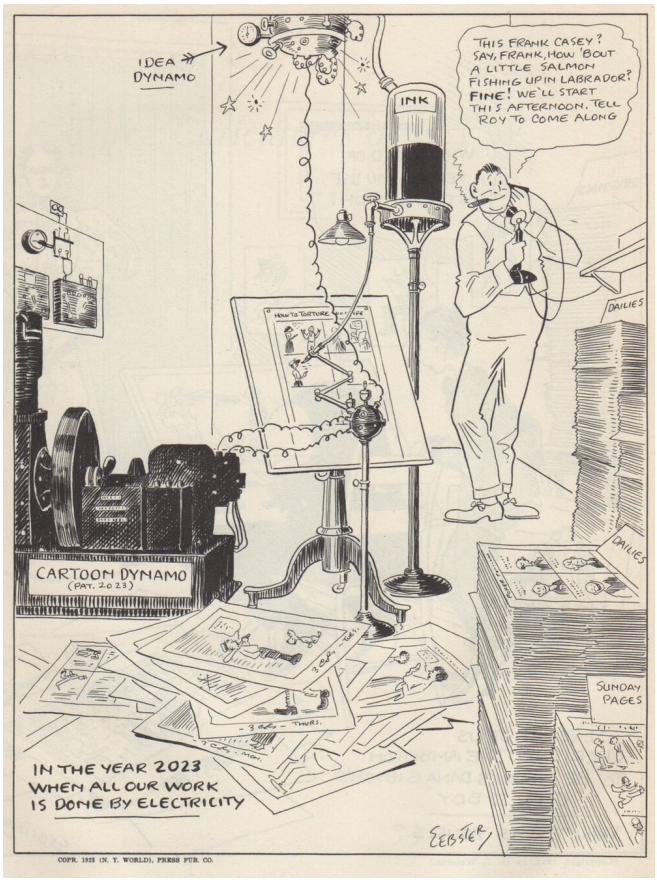


Figure 1: Cartoon by H.T. Webster published in 1922.

2 COURSE BASICS

- Lecture: Monday, Wednesday, Friday 1:50 pm - 2:55 pm
- Lab: Thursday 10:55 am - 12:40 pm
- Location: Olin 107
- Student/Office Hours:
  - Steinmetz 217
  - Monday 3:00 pm - 4:30 pm
  - Thursday 2:00 pm - 3:30 pm
  - subject to change, check course website for most up-to-date schedule
  - drop-in or schedule a 15 minute slot: https://calendar.app.google/3kgVY1utGu5Mtzz7A
  - by appointment for another time or over zoom

3 OVERVIEW

Can computers think? Is it possible to build intelligent machines? What does it mean for a machine to think or to be intelligent? Why, or why not, would we want intelligent machines? To what extent do we already have them? How do they work? How do they impact our lives? What are their limitations? This course invites you to explore these questions and more.

This course is an introduction to the field of computer science with an artificial intelligence theme that does not assume you have prior experience with computer programming or computer science. It introduces basic algorithms, data structures and programming techniques. You will learn how computer scientists think about and solve problems by doing your own computer programming. This will give you an understanding of how the current technology for building intelligent machines works. In addition, you will read non-technical papers about artificial intelligence and the ethical questions raised by its use. You will reflect on the ways current computational systems often reinforce biases and can harm groups of people inequitably. And you will evaluate how the choices you are making when creating your own programs will affect future users.

If you're thinking about a CS or CPE major or minor, this course will give you a solid foundation to continue to *Programming on Purpose* (CSC 120), to *Data Structures* (CSC 151) and beyond. If you're a neuroscience major, it will help you see how scientists are using biological and neurological principles to model "behavior' ' in computers and it will teach you skills useful for processing experimental data. If you are thinking about another major or minor, this course will give you foundations to apply computing in whatever area you are interested in. And if you're here just because you're curious, that's great!

4 LEARNING OBJECTIVES

By the end of the course, you should be able to *answer* the following questions:

- What are the most important programming fundamentals and why are they the building blocks of programs?
- What are the debugging fundamentals and how do you use them?
- What are some of the mechanisms used to build "intelligent" machines?

By the end of the course, you should be able to *do* the following:

- Use variables, functions, conditionals, loops and recursion
- Read Python programs
- Write short Python programs that solve a given problem
- Fix short Python programs using debugging fundamentals

Computing/programming topics we will cover include functions, variables, assignment, conditionals, loops, recursion, lists/arrays, file reading/writing, binary numbers and ASCII, strings and string methods and dictionaries.

A 1-page schedule-at-a-glance is included at the end of the syllabus, but please see Nexus for a link to a more detailed schedule.

5 CLASS POLICIES AND GUIDELINES

The Computer Science Department as a whole welcomes all people, regardless of age, background, beliefs, ethnicity, gender identity, gender expression, national origin, religious affiliation, sexual orientation and any other differences, be they visible or non-visible. It's also important to recognize that institutional racism has prevented members of marginalized groups - especially black, indigenous and other people of color (BIPOC) - from fully participating in the field of Computer Science.

You. Yes you, the one reading this syllabus. You belong here.

As an instructor, I will do my utmost to uphold these principles and to treat everyone with respect. As students in this class, I expect you to do the same since one person is not a community unto themselves. We're all in this together, so let's treat each other well. Lastly, I welcome feedback on issues we might discuss or ways that our class can be a more just one for all people.

6 COURSE MATERIALS

Online Textbook: Foundations of Python Programming Functions First - Interactive (free).

We are going to use an online, interactive, free and open source textbook. This textbook has a lot of embedded interactive exercises. Reading this book always also means doing the activities. To get access to the book, please register on Runestone using the following informationm which can also be found on Nexus:

- https://runestone.academy/runestone/default/user/register
- your Union College email
- unioncollege_foppff_fall26 as the course name

We'll use the following websites and software throughout the course:

- The course website: a github-hosted site with assignments, materials and announcements
  - https://georgiadoing.github.io/CSC106-F26/
  - you may want to bookmark this to check it before and after each class
- Gradescope: used to submit programming assignments
  - https://www.gradescope.com/courses/1370118
  - Entry Code: 3ZGEVN
- Python 3.13.X: the software package needed to write and run Python programs
- Reeborg's World: a useful website for learning some programming basics
- https://reeborg.ca/index_en.html
- Python Tutor: a useful website for tracing python code
  - https://pythontutor.com/visualize.html#mode=edit

7 COURSE RESOURCES

The course website has a list and links to the readings and software resources that we are going to use. In class, we are going to use the Linux machines in Olin 107. Outside of class you have a choice or hardware. You can install the software on your own computer (Python is freely available for Windows, Mac, and Linux, Reeborg's World and Python Tutor are browser based programs) or you can work in one of the Computer Science labs. We have three spaces that you can use 24/7 using your ID card, except when classes are being held in them:

- Olin 107 - where we have class
- PASTA Lab (ISEC 051+)
- Computer Science Resource room (Steinmetz Hall, 209A)

8 LIBRARY RESOURCES

The library is available to help you with your research needs! Librarians can help you develop research questions, search for and select the best sources for your projects, identify research strategies, evaluate sources, and assist you with creating citations. There are multiple ways for you to contact a librarian. For more information, please see our Ask A Librarian page: https://www.union.edu/schaffer-library/ask-a-librarian.

9 ACCOMODATIONS

It is the policy of Union College to make reasonable accommodations for qualified individuals with disabilities. If you are a person with a disability and wish to request accommodations to complete your course requirements, please make an appointment with me or stop by during my office hours as soon as possible, all discussions will remain confidential. You must provide reasonable notice and be in touch with Accomodative Services on the 2nd floor of Schaffer Library, if you have not already, if you wish to take advantage of extra time on exams.

10 COMPLEX QUESTIONS: GLOBAL CHALLENGES & SOCIAL JUSTICE

Justice, Equity, Identity, Difference (JEID). Algorithms and computational systems increasingly shape the world we live in. Advances in AI and machine learning are in the news on a daily basis. But there is also a growing awareness of the limitations and dangers of these systems. In particular, they often promote biases and inequalities. For example, facial recognition software has been shown to work much less accurately for people of color than for white people, an AI hiring algorithm was found to reject female applicants for technical positions at a higher rate than male applicants, and search results can be full of racial stereotypes. In this course, you will learn about real-world cases of AI systems that discriminate against groups of people. You will see that related ethical questions are raised even in the context of the relatively simple programs you create in an introductory computer science class. Additionally, you will work on developing a habit of always evaluating how your own choices as a programmer might impact other people's experiences.

Engineering, Technology and Society (ETS). In this course, you learn to break down a larger problem into a series of smaller computational steps (creating an algorithm) and to implement algorithms in Python. Through these activities you will gain an understanding of how computers work and an idea of the software development process. In particular, you will learn about iterative development and the importance of thorough testing. You will practice communicating about algorithms and their implementation in groups.

You will also explore, through assignments and discussion, how computational systems can have unintended and negative consequences for groups of people and what the responsibility of the programmer is to mitigate these dangers.

Data & Quantitative Reasoning (DQR). Much of current AI systems are data driven and learn based on collections of data. In this course, you will learn about and implement some approaches that AI systems use to draw inferences based on data. Many of the ethical issues around current AI systems are linked to the data these systems use. Throughout the course, you will, therefore, also reflect on: What information is collected? How is the information represented? Who is represented in the data? How is it used?

11 GRADING

The following components will contribute to your final grade for this class with roughly the indicated weight.

- 2 practical exams: 20% (10% each)
- Written midterm: 20%
- Written final exam: 20%
- Programming project demos: 20%
- Portfolios & share-outs: 15%
- Reading & practice: 5%

Note: You must get a C- or better in order to take any other class that has a CSC-10x prerequisite. Several larger programming projects will be assigned to reinforce the concepts discussed in class and put into practice in homework assignments. All projects can be completed using programming techniques that have been covered in class.

Do not use techniques that have not been covered in class unless specifically told otherwise. Part of the challenge for each programming project is figuring out how to complete the assignment using only the tools provided in the course up to that point.

Letter Grade Scale:

- A: 93-100; A-: 90-92
- B+: 87-89; B: 83-86; B-: 80-82
- C+: 77-79; C: 73-76; C-: 70-72
- D: 60-69; F: 0-59

12 ATTENDANCE

Class participation is a critical component of the course and attendance is mandatory. I realize that sometimes other things come up (interviews, athletic events, illness, emergencies etc.). In those cases, just let me know (in advance, if at all possible) that you are going to be absent. There may not always be the possibility of make-up credit for missed in-class material (share-outs), but it is more likely for something to be arranged if you discuss your absence with me prior to class. No matter how much class you have missed, please do not come to class if you have an illness that is likley contageous, please email me to work out a reasonable solution.

If you do miss class, it is your responsibility to make up any material that you missed. Get notes from a classmate, make sure to complete all assignments due or assigned during the class that you missed, and come see me if you have any questions on the material. I will expect you to hand in all assignments by the due date, regardless of whether you were in class. You will not be able to make up missed exams or in-class share-outs without pre-approved exceptions.

13 LATE POLICIES

Portfolio assignments will usually be discussed in class on the day that they are designated, portfolios will be collected the day they are due and will not be accepted late. Programming projects are due by 11:59:59pm on the due date. You have three "extension tokens" for this class. Each token will extend a single project deadline by 24 hours. You can use each token on a different project, or use two or even all three on a single project. Once all three tokens have been used, no late submissions will be accepted (barring exceptional circumstances).

14 PARTICIPATION AND ENGAGEMENT

Following these guidelines will constitute positive participation and engagement and earn you **up to 1.5% of your final grade per week**. To demonstrate comprehension of in-class and/or homework assignments and earn relevent engagement points through share-outs (0.5% per week), students will submit written answers for collection as portfolios (1% per week) and adhere to the follwoing guidelines. **Falling short of any of the below standards may result in point deductions from any engagement activities relevent to class time(s) during which the student was unengaged**.

- Respect. This course is a space for rigorous and respectful debate. When confronting ideas and people different from you, lead with curiosity rather than judgement. We will

critique ideas, not people. To protect our shared learning environment, you may not record, photograph, or share any part of our class sessions outside of our class.

- Engage. Show up to class on time and prepared. Show up ready to learn by completing the readings and exercises and checking the course website often. Engagement in class constitutes being present, respectful and lending the class your full attention. **It is the student's responsibility to demonstrate this engagement by completing and handing-in thoughtful written responses as directed by in-class prompts and/or homework assignments.** Accessing off-topic websites or software, checking email or phones, *utilizing large language models* or otherwise attending to things not pertinant to the course is distracting to yourself and others and will result in point deductions.
- Embrace intellectual humility. Recognize that there are limitations to your knowledge and that some of your beliefs could actually be wrong. Be curious about your thinking and open to learning from others. This is hard, but so vital to your learning in this course—we'll work on developing this throughout the term!
- Get help when you need it. If you are stuck or confused or lost, be proactive and get help. The best learners and thinkers can figure out when they are stuck and make a plan to get unstuck. Come to Office Hours (Steinmetz 108B), the CS Helpdesk (Sun-Thurs 7:00-9:00pm Olin 107) or ask another student. Often the best person to explain something is the person who just figured it out. Try reaching out to another student in the course who seems like they get it—they will probably be flattered you asked!

15 ACADEMIC INTEGRITY AND "ARTIFICAL INTELLIGENCE" USE

Union College recognizes the need to create an environment of mutual trust as part of its educational mission. Responsible participation in an academic community requires respect for and acknowledgement of the thoughts and work of others, whether expressed in the present or in some distant time and place. Matriculation at the College is taken to signify implicit agreement with the Academic Honor Code, available at: http://muse.union.edu/honorcode

It is each student's responsibility to ensure that submitted work is their own and does not involve any form of academic misconduct. Students are expected to ask their course instructors for clarification regarding, but not limited to, collaboration, citations, and plagiarism. Ignorance is not an excuse for breaching academic integrity. Students are also required to affix the full Honor Code Affirmation, or the following shortened version, on each item of coursework submitted for grading:

'I affirm that I have carried out my academic endeavors with full academic honesty.'

[Signed, Jane /John /Jaden Doe]

What it means for this course: **you must complete the programming projects on your own, but you are welcome and encouraged to collaborate on other regular assignments.**

Please notice that different rules apply to the exercises that are part of the textbook, regular assignments and programming projects.

Textbook Readings and Regular Exercises:

You are welcome to discuss the textbook readings, the exercises embedded in the textbook, and other regular assignments with other people in the class to help you fully understand the material. However, don't just copy somebody else's solution. Even if you ask other people for help, the goal is for you to understand the underlying concepts and logic. It's more important to understand how to arrive at a correct solution than it is to know the solution itself.

Programming Projects:

Any work you turn in has to be your own. The best way to learn programming is by doing programming. I expect that you already know that if you do not do things for yourself, you will not learn them. Sometimes that will mean that you will have to struggle. That is ok: struggling to figure out how to solve a problem is where a lot of learning happens. Give yourself the opportunity to do this.

Please, do not ever copy any part of a solution from a friend, from any material generated by an "artificial intellence" (AI) chatbot model (ChatGPT, Gemini, Copilot, Claude, etc.) or from anywhere else. Do not ever *look at* somebody else's solution or any solution generated by an entity other than yourself before you have completed your own. Do not ever *show* somebody else your solution before you've both submitted it; though it may be beneficial to discuss your solution with another student in the class after you've both handed in the assignment. Please do not *ever* show your solution to future students of this class or any chatbot, and do not *ever* post your solutions on the Internet.

Don't search for information on the Internet. Don't ask for information from any AI model. Especially, don't look at any material that provides solutions to exercises that are similar to the projects. An exception is, of course, any online material that is linked directly from the Nexus page. As always, you should practice good citation behavior and cite any sources that you consult in your work (for example, which pages from the Python documentation website you consulted when working on the project).

Discussing your work (without looking at code): Talking through a problem is extremely beneficial in understanding the problem and coming up with a solution path. So, I strongly encourage you to talk to other students in this class about the projects, visit the CS Helpdesk and come to my Office Hours (you can also request a meeting if you have schedule conflicts with my Office Hours).

Here are some ways in which you can talk about the projects without having to worry about violating the rules from above.

- It's ok (and strongly encouraged!) to draw **memory diagrams** together. Draw memory diagrams and discuss possible algorithms for solving the problem.

- It's ok to **write down an algorithm in English** together. Then go off by yourself to implement it into code.
- It's ok to read and write code together that is not part of the assignment, but that helps demonstrate the concept of what you're being asked to do. Go through an example from class or from the book together, or **come up with your own examples**. There's no better way to understand something than trying to think up examples in order to teach it to someone else. Try it!

Do not work on the implementation together with friends, especially if you are each at your own computer and are always in lock step trying to figure out the same line or section of code at the same time. Have a higher-level discussion (using one of the strategies suggested above), then work on the implementation on your own.

Your goal for discussions about any assignment should be that you come away with a better understanding of the problem and of possible ways to approach it so that you can then try out these approaches on your own. You should never leave a discussion with just an answer, without an understanding of how to arrive at that answer. A good general guideline for any discussion, or interaction with an AI model, is that you should not leave the discussion with anything written or typed.

16 CONTACTING ME OUTSIDE OF CLASS

The best method for contacting me outside of class is to stop by my office during Office Hours or whenever my office door is open. You can also schedule a time with me if my regular office hours don't work. Please contact me via email (doingg@union.edu) and we can arrange a phone or zoom call. I respond to emails as soon as possible, but it may take up to one day to get a response during the term, and longer between terms.

17 ADDITIONAL RESOURCES

Mental Health and Campus Resources

Union College is committed to supporting and advancing the mental health and well-being of our students. During the course of their academic careers, students often experience personal challenges that contribute to barriers in learning, such as drug/alcohol problems, strained relationships, chronic worrying, persistent sadness or loss of interest in enjoyable activities, family conflict, grief and loss, domestic violence, difficulty concentrating, problems with organization, procrastination and/or lack of motivation. Students also sometimes come to college with a history of learning difficulties (e.g., any form of special education), experience difficulties succeeding in a particular subject (e.g., math, reading), or have experienced some form of trauma be it emotional or physical (e.g., head injury). These mental health concerns

can lead to diminished academic performance and can interfere with daily life activities. If you or someone you know has a history of mental health concerns or if you are unsure and would like a consultation, a variety of confidential services are available. The Eppler-Wolff Counseling Center provides free counseling and therapy services (including psychiatry) to all enrolled students. Please call (518) 388-6161 or visit the Wicker Wellness Center in person any weekday between 8:30 and 5:00 to schedule an initial contact appointment. Visit the Counseling Center website for more information.

In a crisis situation, or after hours, contact Campus Safety at (518) 388-6911. The National Suicide Prevention hotline also offers a 24-hour hotline at (800) 273-8255.

Any student who faces challenges securing their food or housing and believes this may affect their performance in the course is urged to contact the Dean of Students (dos_office@union.edu) for support or drop into office hours Monday - Friday 8:30 am - 5:00 pm in the Reamer Campus Center room 306. Furthermore, please notify me if you are confortable doing so and I will provide any resources I can. The Union College Persistance Fund can provide financial support in times of unexpected harship to cover needs such as emergency medical expenses not covered by insurance and basic living expenses. Additional sources of supoprt are outlines on the Dean of Students webpage: https://www.union.edu/dean-students .

18 TENTATIVE SCHEDULE

W	TOPIC	Due by end of week	Runestone
1	Programming, Syntax and Semantics	Portfolio (PF) 1	1 & 2
2	Variables and Functions	PF 2	3 & 4
3	Conditional Statements	PF 3, Project Demo 1	5
4	Conditional Loops	PF 4, Practical Exam 1	6
5	Counting Loops	PF 5, Project Demo 2	7
6	Strings, Lists	PF 6, Written Exam 1	8 & 9
7	Nested Lists, Dictionaries	PF 7, Project Demo 3	10
8	File I/O	PF 8, Practical Exam 2	11
9	Recursion	PF 9, Project Demo 4	
10	Searching, Sorting	PF 10	
	Finals Week Monday 11/23 2:30 - 4:30	Written Exam 2	

18.1 ADDENDA

All updated text in this syllabus will supersede text from older versions.

19 PROGRESS TRACKER

Learning Goals	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10
What are the most important programming fundamentals and why are they the building blocks of programs?										
What are the debugging fundamentals and how do you use them?										
What are some of the mechanisms used to build “intelligent” machines?										
Use variables, functions, conditionals, loops and recursion.										
Write short Python programs that solve a given problem.										
Fix short Python programs using debugging fundamentals.										

Grading

Table 2: Grading Scheme

Course Element	Grade Point Contribution	Notes
Reading & Practice	5%	Runestone Interactive Textbook
Portfolios & Share-outs	15%	From in-class and take-home problems
Programming Project Demos	20%	4 at 5% each
Practical Exams	20%	2 at 10% each
Midterm	20%	
Final Exam	20%	written
total	100%	

Table 3: Letter Grade Scale

Letter Grade	Percent range
A	93 - 100
A-	90 - 92
B+	87 - 89
B	83 - 86
B-	80 - 82
C+	77 - 79

Letter Grade	Percent range
C	73 – 76
C-	70 – 72
D	60 – 69
F	0 – 59

Note: You must get a C- or better in order to take any other class that has a CSC-10x prerequisite. Several larger programming projects will be assigned to reinforce the concepts discussed in class and put into practice in homework assignments. All projects can be completed using programming techniques that have been covered in class.

Do not use techniques that have not been covered in class unless specifically told otherwise. Part of the challenge for each programming project is figuring out how to complete the assignment using only the tools provided in the course up to that point.

Assignments

PF 01

CSC106: Portfolio 1

- due Wednesday 9/16/26 by class time
- Intro Survey
- Runestone Chapters 1 & 2
  - https://runestone.academy/runestone/default/user/register
  - your Union College email
  - unioncollege_foppff_fall26 as the course name
- Do your share-out
  - check the schedule to see what problem and what date you will share:
  - add your solution to the share-out slide-deck
- Finish all parts of the Week 1 Portfolio
  - especially note parts we won't start in class and you will have to do on your own ("oyo")
  - there may also be parts we planned to start in class but run out of time for… regardless of the planned in-class start and share-out times, you are responsible for finishing the portfolio by the due date
  - Starter code: winding_sandbar.json

NAME:

STUDENT ID:

Remember to also initial in the upper right-hand header of each page.

Due Wed. Sept. 16th by the end of class.

Portfolio 1: Programming Syntax and Semantics

To succeed in these problems you will need to read and engage with the embedded activities in the following **Runestone texbook chapters** (worth 0.5 of a grade percentage point):

- Ch. 1: Introduction
- Ch. 2: Variables, Statements and Expressions

For completing and self-correcting 70% of the below activities (during in-class share-outs), you will earn 1 grade percentage point. For participating in a *share-out of 1 of the activities, you will earn 0.5 of a grade percentage point*.

Note that some activities we will plan to start in class and others you need to start on your own ("oyo"). You should finish each activity before its in-class share-out in order to correct it before final submission. Note the intended schedule below but keep in mind that the schedule is always subject to change.

Remember to print and attach your programs for the coding activities. Programs should be submitted to gradescope for autograder results, but credit is given to programs printed and included in this portfolio, on completetion, regaurdless of autograder results. Programs should be marked up for corrections during in-class share-outs.

Table of Contents

This portfolio includes the following 10 activities and the report of your weekly share-out:

A1) Prompt: Can computers think? Why or why not? What does it mean, to "think"?

A2) Programs for Human Turtles

1. Fold this sheet into thirds & turn it over.
2. On the back top third, draw something that could have been drawn by the Turtle robot.
3. On the front, in the top two thirds, write instructions (in English) for the robot to create this drawing.
4. Swap your instructions with someone who didn't see your drawing.
5. **Without looking at the drawing on the back** draw a picture following the instructions.
6. Fold down the top third and see - how closely does your drawing match the original?

……

instructions:

……

leave blank for drawing:

……

Draw something a turtle-bot could draw:

……

A3) Computers, Algorithms and Programs

- Take a look at **Runestone Chapters 1 & 2** … feel free to look at it while answering these questions and during today's discussion

0. Name some parts of a computer.

   ……

   ……

   ……

   ……

1. Where are the programs you write stored when the computer is powered off? Why?

   ……

   ……

   ……

   ……

2. Where are the programs currently running on a computer stored? Why?

   ……

   ……

   ……

   ……

3. What is a reserved word in Python? Name a few.

   ……

   ……

   ……

   ……

4. How would you describe the difference between an interpreted programming language and a compiled programming language?

   ……

   ……

   ……

   ……

5. Describe some pros and cons of interpreted and compiled programming languages.

   .....

   .....

   .....

   .....

6. Can more than one algorithm be used to solve a problem? Why or why not?

   .....

   .....

   .....

   .....

7. Can an algorithm be written in more than one programming language? Why or why not?

   .....

   .....

   .....

   .....

8. What is a function? Why might functions be useful?

   .....

   .....

   .....

   .....

9. What is a parameter?

   .....

   .....

   .....

   .....

10. What is an argument?

    .....

    .....

    .....

    .....

A4) Programs for Robots: Working with Reeborg

Paired Programming Instructions

- Pair programming is a widely utilized programming method popular in industry and academia for the increase in quality it gives to both process (learning, comprehension, memory) and product (algorithm, program, results).

Paired programming guidelines:

- Roles:
  - driver: has the keyboard and types
  - navigator: guides the driver
- Instructions:
  - You should be talking to each other constantly!
  - You should be able to swap roles at any moment.
  - Swap roles every 5 to 10 minutes.

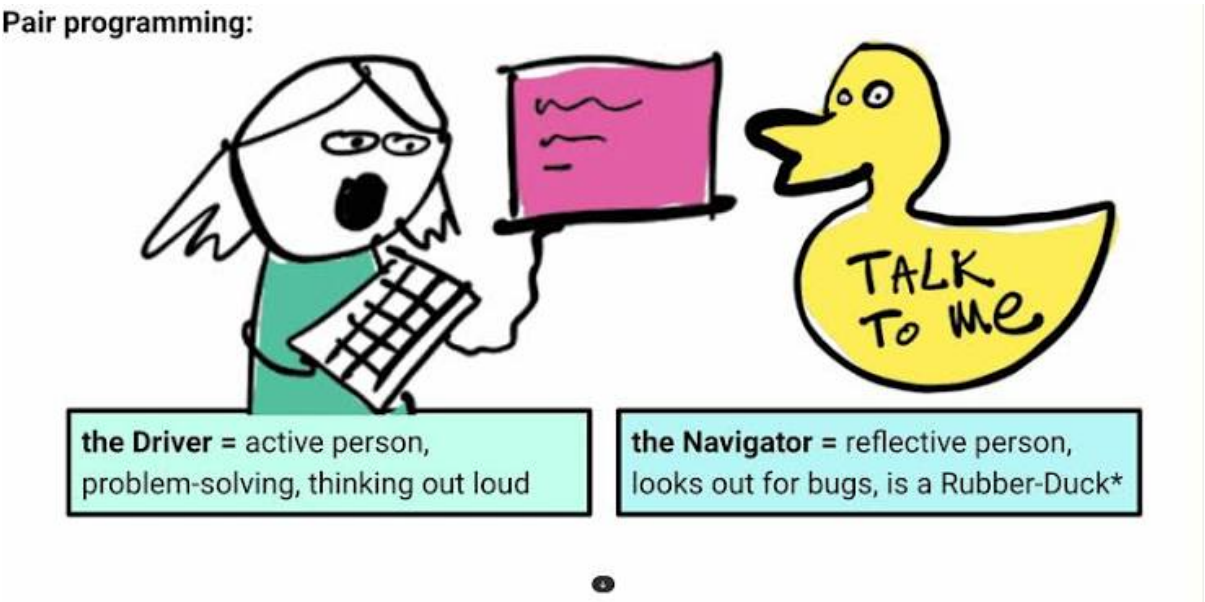
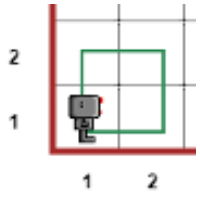


Figure 1: Pair Programming

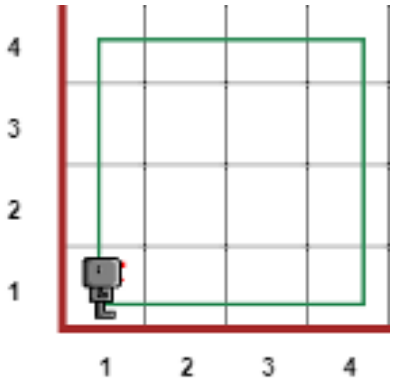
- Note that most programming we do in class will be pair programming. The exceptions will be in-class practical exams where each student will work independently.
- You will have the opportunity to practice indeipendent programming when finishing up assignments you don't finish in class, doing on assignments we don't start in class and doing your projects.

From the course website, under Resources, follow the link to Reeborg's World, or type out the url: https://reeborg.ca/reeborg.html

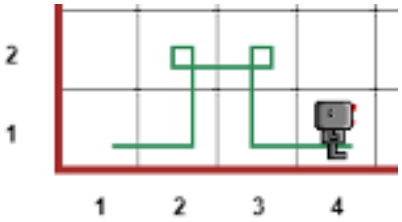
1. Write programs to make Reeborg walk the following patterns. Save each program in separate files.



(a) Reeborg_pattern1



(a) Reeborg_pattern2



(a) Reeborg_pattern3

Reeborg patterns

2. In english (not code), what is your algorithm for each pattern?

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

- To save, choose "Save program to file" from the "Program in editor" section of the "Additional options" menu.
- Note: Python programs are saved as text files, but the file names have the extension `.py`

3. Upload each file to your Google drive, or email to yourself, so you can print out these files to submit with your portfolio.

For portfolio submission, insert you printed program pages here.

A5) Programs for Robots: Delivering a newspaper

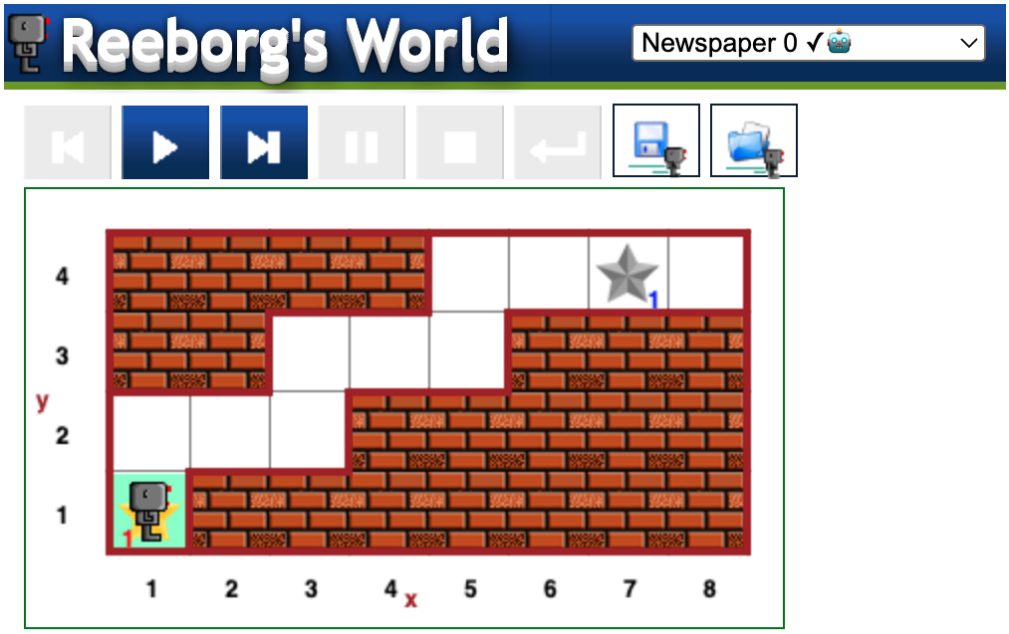


Figure 5: Reeborg Newspaper 0

1. Select the `Newspaper 0` world. It should look like the image above. Find the "World Info" button, just to the right of the upper right hand corner of the above screenshot. What is the goal in this world?

   .....

   .....

   .....

   .....

2. In english (not code), what is your algorithm for your solution?

   .....

   .....

   .....

   .....

   .....

   .....

3. Write a program to implement your algorithm and solve `Newspaper 0`.
4. Add comments to your program to explain what it's doing (each step of your algorithm).
5. Save the program in a file called `newspaper0.py`

6. Submit `newspaper0.py` on gradescope:

    a. Follow link to gradescope (link can be found on the course website)
    b. Select Union College from the list of options
    c. Login using your Union College credentials
    d. Find the assignment: `In-class Assignment: Newspaper 0`
    e. Upload `newspaper0.py` to the assignment

7. Did the autograder produce error messages? How might you revise your algorithm and/or program?

    .....

    .....

    .....

    .....

    .....

    .....

    .....

8. Were you able to solve this task? If not, can you think of anything else to try?

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

9. Share the code you worked on with your partner(s) before you leave class. Upload it to your Google drive or email it so you can each complete it, if not already done, and print it out.

For portfolio submission, insert you printed program pages here.

A6) Check-in Questions, Syntax and Semantics

Answer the following questions about Reeborg and Python (Turtle) syntax. Think about what the program syntax looks like and means. Programs can be valid, meaning they have proper syntax, but to be *correct* they must also do what the programmer intends them to do, meaning the semantics are right.

Take a look at Runestone Chapter 2 before completing the rest of this portfolio.

1. Which of the following programs are valid Reeborg programs that cause Reborg to execute a sequence of actions? What's wrong with the invalid ones? (Assume Reeborg won't run into walls)

   a.
   ```
   Move()
   Move()
   Move()
   ```

   b.
   ```
   move()
   turn_left()
   turn_left()
   turn_left()
   move()
   ```

   c.
   ```
   move()
   move()
   turn_right()
   move()
   ```

   d.
   ```
   move
   turn_left
   move
   move
   ```

2. The following is a function call statement:
   `turn_left()`

   a. What is the name of the function being called?

   .....

   b. What purpose do the parentheses serve?

   .....

   c. What is a function?

   .....

   .....

   d. What does "calling a function" mean?

   .....

   .....

3. Consider the following Python program. Draw the pattern that the turtle draws when this program is executed. Also indicate the position and orientation of the turtle after this program has been executed. Hint: The turtle starts out in the center of the turtle window, facing to the right

```
import turtle

turtle.left(90)
turtle.forward(200)
turtle.left(180)
turtle.forward(100)
turtle.left(90)
turtle.forward(100)
```

4. Which of the following programs are valid Python programs? Indicate which are valid. For the invalid ones, explain or indicate what is wrong. The import statement isn't shown, but you can assume that the turtle module has been imported.

   a.
   ```
   turtle.forward(100)
   turtle.right(90)
   turtle.forward(100)
   ```

   b.
   ```
   turtle.forward
   turtle.left
   turtle.forward
   ```

   c.
   ```
   turtle.forward(100)
   turtle.turn_right(90)
   turtle.forward(100)
   ```

   d.
   ```
   turtle.Forward(100)
   turtle.right(90)
   turtle.Forward(100)
   ```

5. What required the most thinking — coming up with algorithms, writing programs or communicating in pair programming? Did the activities rquire different *types* and/or *amounts* of thinking?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

A7) Tracing (re-)assignment with Memory Diagrams

Consider the following snippet of Python code.

```
1.  x = 5
2.  y = 10
3.  z = x
4.  x = y + 3
5.  y = z
```

1. Draw a memory diagram that shows the <variable name>-<object/value> associations after line 3 has been executed.

2. Draw a memory diagram that shows the <variable name>-<object/value> associations after line 4 has been executed.

3. Draw a memory diagram that shows the <variable name>-<object/value> associations after the whole snippet has been executed.

4. In Python, what happens in the machine's memory blocks when a variable is assigned to another variable? Do the variables share a memory block, or is a new memory location used? Assume the inital variable was assigned to an integer value.

5. Depict the process you described above with a diagram/drawing.

A8) Arithmetic in Python

Computer programming languages are often designed to handle arithmetic operations such as addition, subtraction, multiplication, division, and exponents. These operations are performed through the use of operators, several of which are listed below:

$$+, -, *, /, //, \%, **$$

Just as in mathematics, we can use these operators to develop complex mathematical equations. Also like in math in a more general sense, parentheses can be used to give some operations a higher priority than they would normally have. For example:

$$2 + 4 * 2 = 2 + 8 = 10$$
$$(2 + 4) * 2 = 6 * 2 = 12$$

1. Using IDLE, write a Python program that uses each of the operators shown above at least once. Please include comments explaining what it looks like each operator does. You may need to experiment with some of them to figure out exactly what they're doing. Name your program `arithmetic.py`.
2. Add print statements to give the same information as the comments so a user will see an explanation of what the program is doing whenever they run it.
3. Taxes, tips, and totals: Now we're going to write a program to calculate the tax, tip, and total cost for dining at a restaurant. Name your program `receipt.py`.

   Start by assigning the following values to variables:

   i. The meal's subtotal of $35 (example name: `meal_cost`)
   ii. The meal's tax rate of 7% (example name: `tax_rate`)
   iii. The meal's tip rate of 25% (example name: `tip_rate`)

4. Add code to compute the tax, tip, and total cost for the meal, then use print() to display a receipt that displays the following information:

```
RECEIPT
Sub-total: $35
Tax:       $2.45
Tip:       $8.75
Total:     $46.20
```

5. Make the numbers line up nicely. For the time being, it's perfectly acceptable if $46.20 appears as $46.2 instead.
6. Change the values assigned to each of the three variables you created. Do the results produced by the altered program make sense? (e.g. is the tip computed correctly?) If not, then figure out why and try to fix it.
7. Adjust the program so that `meal_cost`, `tax_rate`, and `tip_rate` are assigned from user input (i.e. using the `input()` function and type-casting).

8. Test the new version of the program to make sure that the receipt information is correct based on the input.
9. If you haven't already done so, make sure to add comments explaining how the program works.
10. Can you display the whole receipt using a single `print()` function call?
11. Did the autograder produce error messages? How might you revise your algorithm and/or program?

    .....

    .....

    .....

    .....

    .....

    .....

    .....

12. Were you able to solve this task? If not, can you think of anything else to try?

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

    .....

For portfolio submission, insert you printed program pages here.

A9) Programming practice: Reeborg

0. Do this part on the Reeborg's World website.
1. Download the starter code from the course website.
    - It is one file called: `winding_sandbar.json`
2. Import the starter file winding_sandbar.json into Reeborg.
    - To import a program, open the "Additional options" menu.
    - Then look for the button "Open world from file."
    - This should show you an environment that looks as follows:

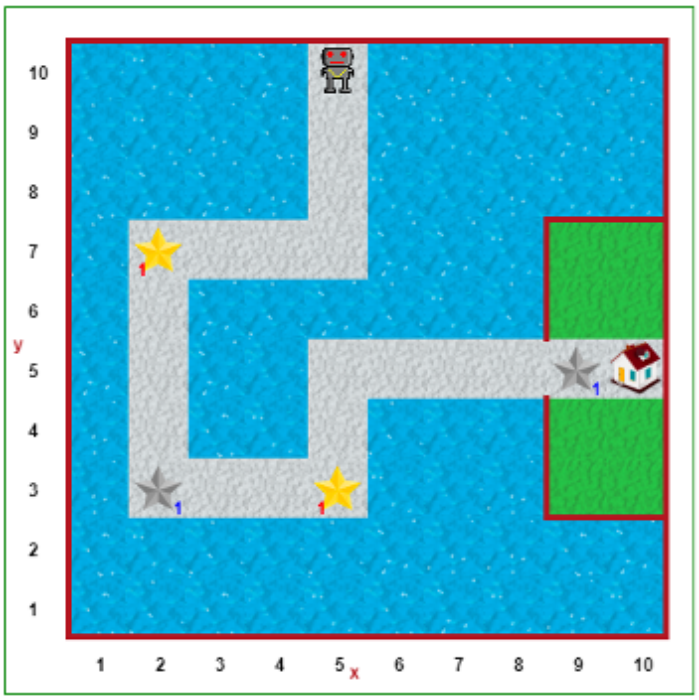


Figure 6: Reeborg Winding Sandbar

3. What is the goal in this world?

.....

.....

.....

.....

.....

4. In english (not code), what is your algorithm for your solution?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

5. Complete the program so that Reeborg picks up the stars, deposits them in the indicated spots and finishes at the house. Save your program file (not the world file) in a file with the name `winding_sandbar.py`.

   - When pair-programming, as you go, add comments throughout indicating who was the "driver" and who was the "navigator" for each chunk.

6. Share the program between pairs using email or Google drive/
7. Individually add comments to your code. Explain the different steps and including a header comment that names your partner.
8. Submit your code on gradescope. Remember to also print it out to submit with your portfolio.
9. Did the autograder produce error messages? How might you revise your algorithm and/or program?

   .....

   .....

   .....

   .....

   .....

10. Were you able to solve this task? If not, can you think of anything else to try?

    .....

    .....

    .....

    .....

    .....

For portfolio submission, insert you printed program pages here.

A10) Programming Practice: Turtle in Python

0. Your goal is to write a program that uses the turtle module to draw the following picture.

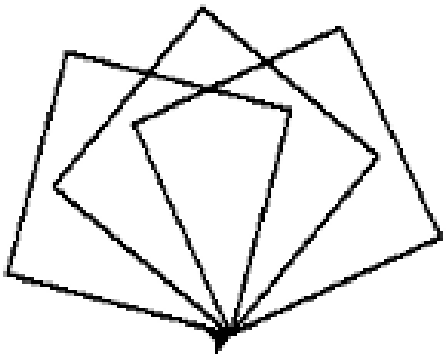


Figure 7: Turtle Squares

1. In english (not code), what is your algorithm for your solution?

   .....

   .....

   .....

   .....

   .....

   .....

2. Now, in IDLE, write your program in Python. Save your program in a file called squares.py
3. Think about how you would modify your program so that it draws the same three squares, but each of them in a different color.

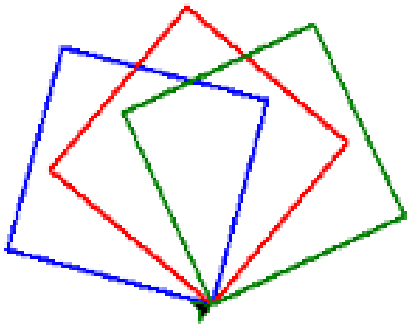


Figure 8: Turtle Squares in Color

3. (cont.) In english (not code), what is your algorithm for your solution?

   .....

   .....

   .....

   .....

   .....

4. Now edit your previous program to implement your new algorithm. Don't forget to add comments to your code to help human readers. Save your program in a file called colored_squares.py.
5. Submit your code on gradescope. Remember to also print it out to submit with your portfolio.
6. Did the autograder produce error messages? How might you revise your algorithm and/or program?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

7. Were you able to solve this task? If not, can you think of anything else to try?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

For portfolio submission, insert you printed program pages here.

B1: Share-out

0. Did you share-out this week? If not, and if you would like to make up your credit, where in the upcoming schedule would you like to be added?

   .....

   .....

   .....

   .....

1. For which activity did you share-out?

   .....

   .....

   .....

   .....

2. What was the solution you shared?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

3. If any, what changes did you make to your solution in response to questions or discussion?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

B2: Share-out 2 (optional, for make-ups)

0. Did you share-out this week? If not, and if you would like to make up your credit, where in the upcoming schedule would you like to be added?

   .....

   .....

   .....

   .....

1. For which activity did you share-out?

   .....

   .....

   .....

   .....

2. What was the solution you shared?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

3. If any, what changes did you make to your solution in response to questions or discussion?

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

   .....

Project 1

CSC106: Project 1

- due Monday 9/14/26 by class time (1:50 pm)
- you may use late token(s) for this assignment

In this project, you'll write code to draw a variety of shapes using the turtle module. Then, you'll use this code to compose a picture and to write a program to randomly generate abstract art.

- Reminder: Programming projects should be done individually. You should not look at anyone else's code, show your code to anyone else, write code for anyone else, or let someone else write code for you. Consider anyone else to include any form of AI. Please see the syllabus or Nexus for what to do when you need help with your work.
- Projects are graded on program correctness and code composition, including proper commenting
- This project is 10% of your final grade
- starter code:
  - square.py
  - turtle_shapes.py
  - computer_art.py

NAME:

Project 1: Drawings with Turtle

In this project, you'll write code to draw a variety of shapes using the turtle module. Then, you'll use this code to compose a picture and to write a program to randomly generate abstract art.

Learning Objectives:

1. Practice designing and implementing simple algorithms.
2. Demonstrate that you can use the programming concepts that we've covered so far: function calls, function definitions, variables and assignment, and counting loops
3. Practice reading the official Python documentation.
4. More practice defining your own functions and using them.

Reminder: Programming projects should be done individually. You should not look at anyone else's code, show your code to anyone else, write code for anyone else, or let someone else write code for you. Consider anyone else to include any form of AI. Please see the syllabus or Nexus for what to do when you need help with your work.

1. Drawing shapes

1.1 Square

1. Open the starter code `square.py`. You'll notice that a few variables are defined at the top of the file. Please use these variables in your code.
2. Add some code that moves the turtle to the (`x`,`y`) position without drawing a line.
3. Add some code that draws the square using size as the side length.
4. Make sure to save your program in a file called `square.py`.
5. Make sure that your code is fully commented.

1.2 Rectangle

1. Create a new code file called rectangle.py using a similar format to `square.py`. Meaning:
    a. Define `x, y` to indicate the starting position for the rectangle.
    b. Create: width and height to control the size of the rectangle.
    c. The entire rectangle should be visible in the turtle window without having to resize it, but beyond this restriction, feel free to place the rectangle wherever you like and to make it as big or small as you like.
2. Add code to move the turtle to position (`x`,`y`).
3. Add code to draw a rectangle with the specified width and height.
4. Save this code in a file called `rectangle.py` and make sure to comment the code.
    - Hint: It may be very helpful to copy and paste code from `square.py` to reduce the amount of typing you have to do.

1.3 Circle

1. Follow the same general steps for the rectangle, except this time you'll be creating a circle.
    a. File name: `circle.py`.
    b. Variables to create: `x, y, radius`.

1.4 Triangle

1. Follow the same general steps as for the rectangle, but this time we're making a triangle.
    a. File name: `triangle.py`.
    b. Variables to create: `x, y, size`.
2. This program should draw an equilateral triangle.
3. The size variable should indicate the length of one side.

2, Functions for shapes

1. Open the starter file `turtle_shapes.py`. This file contains the beginnings of some function definitions. (We'll be discussing function definitions more this week)
2. Copy your code for drawing a square from `square.py` into the `square(x, y, size)` function definition. There are comments explaining how to do this in the starter file.
    - Caution: Do not copy the variable definitions at the top of the square.py file. They will not be needed if you designed your square.py code correctly.
3. You should notice that the `square(x, y, size)` function is called at the bottom of the starter file. Once you've completed the previous step, running the program should produce a drawing of a square in the bottom right quadrant of the turtle window.
4. Repeat step 2 for the other shapes (rectangle, circle, triangle).
    - Caution: Don't change the order of the parameters in the function definitions.
5. Add function calls for each of the shapes at the bottom of the file to test your program and prove that the functions work as expected.
6. Please note that you should copy over comments in addition to the code itself.

3. Draw a picture

1. Create a new file and copy the function definitions from Part 2 into it. Save this new file as `my_picture.py`.
2. Write a program that uses these functions to draw a picture.
3. Picture requirements:
    a. Use at least two line colors and at least two fill colors.
    - Hint: it may be useful to refer back to your in-class assignments. If you're still having trouble, take a look at the `turtle` library section of the Python official documentation (https://docs.python.org/3/library/turtle.html)… or stop by my office hours.
    b. Your picture should depict a recognizable object or scene. For example, something like the picture bloew (though it does not need to be this involved)

c. You may add additional functions to help draw the picture (e.g. functions to draw filled-in versions of each shape), but you should not change the functions that you defined for Part 2 of the assignment.
d. Your program should use each function from Part 2 at least once, but you can also use other built-in turtle functions.
e. Your program should use loops when appropriate to keep your code DRY (Don't Repeat Yourself).

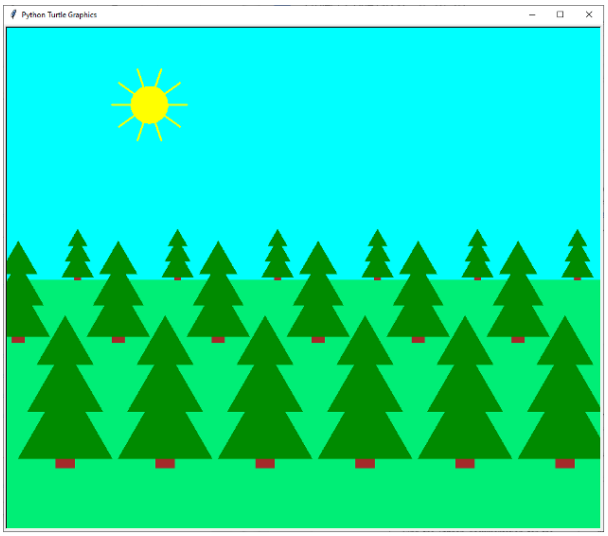


Figure 1: Turtle Drawing

4. Computer Generated art

1. Find the Python documentation for the `random` module. (Hint: Use the link to the "library reference" on the Resources page on the course website)
2. The function `randrange` from the `random` module produces a random integer. Test it out by writing a program that generates five random numbers between 2 and 9 (inclusive) and prints them out. Save your program in a file called `five_random_numbers.py`. Hint: remember to keep your code DRY.
3. Open the starter file `computer_art.py`.
4. This file already contains definitions for two functions. In particular, it contains a function called `random_shape` that randomly picks a shape (square, rectangle, circle, or triangle) and draws it. The function takes three parameters:

    a. `x` - the x location
    b. `y` - the y location
    c. `size` - the size

5. Copy and paste your function definitions from Part 2 into this file.
6. Add some code to draw 30 randomly picked shapes at random locations on the screen. For example, your output could look something like this:

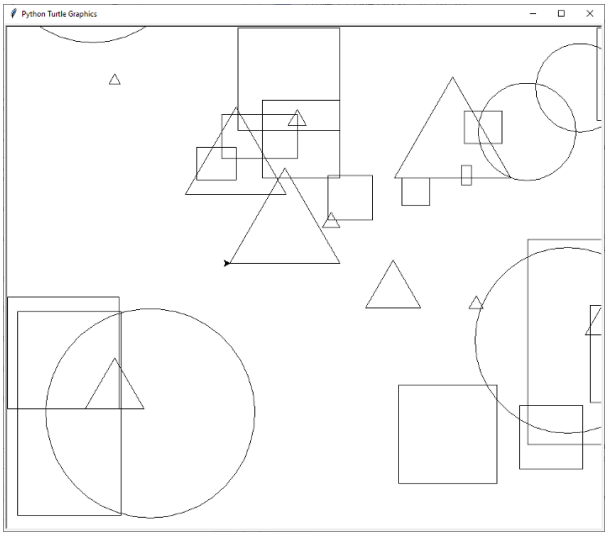


Figure 2: Computer Art

7. (Optional): Add some code to randomly change the pensize, pencolor, and filcolor.

5. What to submit

You will need to submit the following files:

1. `square.py`
2. `rectangle.py`
3. `circle.py`
4. `triangle.py`
5. `turtle_shapes.py`
6. `my_picture.py`
7. `five_random_numbers.py`
8. `computer_art.py`

Before submitting, check that your code meets the following requirements:

1. Are your files properly commented? This should include the following:
    a. A header comment, which includes your name, the date, the assignment and a brief description of the program.
    b. Comments within the body of the code to further clarify how the program works.
2. Are your programs formatted neatly and consistently? Do you use some whitespace to help a human reader understand the logical organization of your code?
    - Hint: it may be helpful to think of this as breaking your code up into "paragraphs."
3. Have you cleaned up any code snippets you no longer need?
    a. The final version of the program should not contain any lines of actual code that are commented out to keep them from running. Such snippets should be removed before submitting.

Once you have checked these things, submit your code to:

"Project 1: Drawings with Turtle" on Gradescope.

6. Grading guidelines

- **Demo specifics to come** … generally speaking, focus on the below elements
- Correctness: programs do what the problems require.
- Program logic: use loops where appropriate, variables where appropriate, and your code doesn't contain lines of code that don't contribute to the program.
- Comments: code is properly commented. There should be a header comment that includes the author's name and describes the program's purpose. Additional comments in the code should help clarify how the program works.
- Organization: use whitespace to indicate the logical structure of the code.
  - Lines of code that logically belong together are close together in the file.
  - Import statements are at the very top…
  - followed by function definitions…
  - then followed by the rest of the code.

Weekly Notes

Week 1

Notes from Week 1

- Wed
  - Seymour Pappert Turtle Documentary
  - Activities for Portfolio 1: A1 (prompt) and A2 (turtle drawing)
- Thurs
  - sort ourselves on "Can computers think"?
  - log onto computers: log on instructions
  - register for Runestone: see Resources page
  - also maybe helpful, a different Runestone book
- Fri
  - Reeborg programs (portfolio A4 and A5)
  - computer basics answers (portfolio A3): share-out

for over the weekend

- on your own
  - questions on syntax vs semantics (PF A6)
  - variable assignment tracing (PF A7)
  - more Reeborg (PF A9), starter code: winding_sandbar.json

look ahead to Monday and next week

- Monday
  - more share-outs
  - first Python program: hello world
  - python practice with arithmetic (PF A8)
- Wednesday
  - more share-outs
  - python practice with Turtle module (PF A10)
  - portfolio collection

About

Me:

- Georgia Doing, PhD
- she/her/hers
- My interests: computational microbiology, small cells & big data (check out my github website for descriptions of research projects)
- My dog Ginny (just email ahead of office hours if you'd like me to make sure she isn't there, no problem!)

References